

Оптимизация моделей компьютерного зрения

Методы ускорения, сжатия и автоматизации

PyTorch · Transfer Learning · NAS · Pruning · Quantization · Distillation

AutoAugment · EfficientNet · MobileNet · ONNX · TorchVision

Тема 1. Зачем оптимизировать модели CV

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Задачи компьютерного зрения (Computer Vision, CV) принципиально отличаются от табличных задач, для которых разработаны классические AutoML-системы. Ключевые особенности:

Во-первых, масштаб моделей. Современные CNN и Vision Transformer содержат от нескольких миллионов (MobileNetV3: 5.4M) до сотен миллионов параметров (ViT-L: 307M). Обучение с нуля требует дней на GPU-кластерах. Transfer Learning и сжатие моделей становятся обязательными инструментами.

Во-вторых, требования к развёртыванию (deployment). Модели CV работают на мобильных устройствах, edge-ускорителях и embedded-системах с ограниченной памятью (< 1 ГБ) и вычислительным бюджетом (< 10 GFLOPS). Pruning, quantization и knowledge distillation позволяют уменьшить размер модели в 4–10 раз без существенной потери качества.

В-третьих, данные. В CV критичны аугментация (AutoAugment, CutMix, MixUp), предобработка (нормализация по ImageNet-статистикам) и выбор разрешения входного изображения — всё это прямо влияет на accuracy и latency.

Семинар охватывает 8 ключевых методов оптимизации: transfer learning, NAS, knowledge distillation, pruning, quantization, аугментация данных, оптимизация гиперпараметров и эффективные архитектуры.

Тема 1. Таксономия методов оптимизации CV

ОБЗОР

Метод	Цель	Эффект	Инструменты
Transfer Learning	Сокращение обучения	×5–50 быстрее	torchvision, timm
NAS	Поиск архитектуры	Лучшая accuracy	DARTS, AutoKeras
Knowledge Distillation	Передача знаний	Маленькая модель ≈ большая	torch, Hugging Face
Pruning	Удаление весов	–50–90% параметров	torch.nn.utils.prune
Quantization	Снижение точности	×2–4 быстрее, ×4 меньше	torch.quantization
Data Augmentation	Расширение данных	+1–3% accuracy	Albumentations, torchvision
LR Scheduling	Оптимизация обучения	Стабильная сходимость	Optuna, torch.optim
Efficient Architectures	Компактные сети	Мобильный деплой	EfficientNet, MobileNet

Тема 2. Transfer Learning и Fine-tuning

❏ ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Transfer Learning — стратегия, при которой модель, обученная на большом наборе данных (source domain), используется как начальная точка для решения задачи на целевом наборе (target domain). В компьютерном зрении source domain — это, как правило, ImageNet (1.2М изображений, 1000 классов).

Теоретическое обоснование: первые слои CNN извлекают универсальные низкоуровневые признаки (границы, текстуры, градиенты), которые переносимы между задачами. Глубокие слои специализируются под конкретный датасет. Поэтому при fine-tuning ранние слои «замораживаются» (freeze), а последние переобучаются.

Стратегии fine-tuning: (1) Feature Extraction — замораживаются все слои кроме классификационной головы; (2) Partial Fine-tuning — размораживаются последние N слоёв; (3) Full Fine-tuning — обучаются все слои с малым learning rate; (4) Discriminative LR — разные скорости обучения для разных групп слоёв.

Библиотека timm (PyTorch Image Models) предоставляет доступ к 700+ предобученным архитектурам (ResNet, EfficientNet, ConvNeXt, Swin Transformer и др.) через единый API: `timm.create_model('efficientnet_b0', pretrained=True)`.

Тема 2. Fine-tuning в PyTorch

код

```
import torch
import torch.nn as nn
import torchvision.models as models

# Загрузка предобученной ResNet-18 (imageNet weights)
model = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)

# Стратегия 1: Feature Extraction – замораживаем все слои
for param in model.parameters():
    param.requires_grad = False # заморозка весов

# Заменяем классификационную голову под наше число классов
num_classes = 10 # например, CIFAR-10
model_fc = nn.Sequential(
    nn.Linear(512, 256), # промежуточный слой
    nn.ReLU(),
    nn.Dropout(0.3), # регуляризация
    nn.Linear(256, num_classes) # выходной слой
)

# Стратегия 2: Partial Fine-tuning – размораживаем layer4
for param in model.layer4.parameters():
    param.requires_grad = True # обучаем последний блок

# Оптимизатор с разным LR для разных частей модели
optimizer = torch.optim.Adam([
    {'params': model.layer4.parameters(), 'lr': 1e-4},
    {'params': model_fc.parameters(), 'lr': 1e-3}
])
```

Тема 2. Задача А: Fine-tuning на CIFAR-10

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Fine-tune предобученную ResNet-18 на CIFAR-10 (10 классов). Сравните accuracy при Feature Extraction и Full Fine-tuning за 5 эпох.

```
import torchvision.transforms as T
from torchvision.datasets import CIFAR10
from torch.utils.data import DataLoader

# Предобработка под ImageNet-статистики
transform = T.Compose([
    T.Resize(224),      # ResNet ожидает 224x224
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], # ImageNet mean
                 std=[0.229, 0.224, 0.225]) # ImageNet std
])

# Загрузка CIFAR-10
train_ds = CIFAR10(root='./data', train=True,
                   transform=transform, download=True)
test_ds = CIFAR10(root='./data', train=False,
                  transform=transform)
train_dl = DataLoader(train_ds, batch_size=64, shuffle=True)
test_dl = DataLoader(test_ds, batch_size=64)

# Цикл обучения (5 эпох)
criterion = nn.CrossEntropyLoss()
for epoch in range(5):
    model.train()
    for images, labels in train_dl:
        out = model(images)
        loss = criterion(out, labels)
        optimizer.zero_grad(); loss.backward()
        optimizer.step()
```

Тема 3. Neural Architecture Search (NAS)

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Neural Architecture Search (NAS) — автоматический поиск оптимальной архитектуры нейронной сети для заданной задачи. В отличие от AutoML для табличных данных, NAS оперирует пространством архитектурных блоков: типы свёрток, число каналов, skip-connections, нормализации.

Ранние подходы (Zoph & Le, 2017) использовали reinforcement learning и требовали 500 GPU-дней для поиска. Современные методы значительно эффективнее:

DARTS (Differentiable Architecture Search, Liu et al., 2019) — relaxation дискретного поиска в непрерывное пространство через softmax-взвешивание операций. Поиск занимает 1–4 GPU-дня вместо сотен.

ENAS (Efficient NAS, Pham et al., 2018) — разделение весов между кандидатами через единый DAG (суперсеть). Контроллер (RNN) выбирает подграф.

ProxylessNAS (Cai et al., 2019) — поиск непосредственно на целевом hardware (GPU, CPU, мобильный NPU) с учётом latency как штрафа в loss-функции.

One-Shot NAS — обучение одной суперсети, из которой извлекаются подсети без переобучения. Используется в EfficientNet, OFA (Once-for-All).

Тема 3. AutoKeras: NAS для CV

код

```
import autokeras as ak # pip install autokeras
import tensorflow as tf

# Загрузка CIFAR-10 через Keras
(x_train, y_train), (x_test, y_test) = \
    tf.keras.datasets.cifar10.load_data()

# AutoKeras: автоматический поиск архитектуры CNN
clf = ak.ImageClassifier(
    max_trials=10, # макс. число архитектур для проверки
    overwrite=True # перезаписать предыдущие результаты
)

# Запуск NAS: поиск архитектуры + обучение лучшей
clf.fit(
    x_train, y_train,
    epochs=10, # эпох на каждую архитектуру
    validation_split=0.2 # валидационная выборка
)

# Оценка лучшей найденной архитектуры
test_loss, test_acc = clf.evaluate(x_test, y_test)
print(f'Test accuracy: {test_acc:.4f}')

# Экспорт найденной модели в Keras
best_model = clf.export_model()
best_model.summary() # структура найденной архитектуры
```

Тема 3.1. Grid Search — полный перебор

код

```
from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.datasets import load_breast_cancer

# Загрузка данных для экспериментов
X, y = load_breast_cancer(return_X_y=True)

# Определяем сетку гиперпараметров для перебора
param_grid = {
    'n_estimators': [50, 100, 200], # число деревьев
    'max_depth': [5, 10, 20], # глубина дерева
    'min_samples_split': [2, 5, 10] # мин. объектов для разбиения
}
# Всего: 3 × 3 × 3 = 27 комбинаций × 5-fold CV = 135 обучений

grid = GridSearchCV(
    estimator=RandomForestClassifier(random_state=42),
    param_grid=param_grid,
    cv=5, # 5-fold кросс-валидация
    scoring='f1', # метрика оценки
    n_jobs=-1, # параллельное выполнение
    verbose=1 # прогресс в консоль
)
grid.fit(X, y)

# Лучший результат и параметры
print(f'Grid Search лучший F1: {grid.best_score_:.4f}')
print(f'Лучшие параметры: {grid.best_params_}')
```

Тема 4. Knowledge Distillation в PyTorch

код

```
import torch, torch.nn as nn, torch.nn.functional as F

def distillation_bss(student_logits, teacher_logits,
                    labels, T=4.0, alpha=0.7):
    """Комбинированный bss: CE + KL-дивергенция."""
    # Мягкие вероятности с температурой T
    soft_student = F.softmax(student_logits / T, dim=-1)
    soft_teacher = F.softmax(teacher_logits / T, dim=-1)

    # KL-дивергенция между мягкими распределениями
    kl_bss = F.kl_div(
        soft_student, soft_teacher,
        reduction='batchmean') * (T * T) # масштабирование

    # Стандартный CE-bss на ground truth метках
    ce_bss = F.cross_entropy(student_logits, labels)

    # Итоговый bss: баланс teacher/ground-truth
    return alpha * kl_bss + (1 - alpha) * ce_bss

# Пример использования в цикле обучения
teacher_model.eval() # teacher в режиме inference
with torch.no_grad():
    teacher_out = teacher_model(images)
student_out = student_model(images)
bss = distillation_bss(student_out, teacher_out, labels)
bss.backward()
```

Тема 4. Задача Б: дистилляция ResNet→MobileNet

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Обучите MobileNetV2 (student) на CIFAR-10 с дистилляцией от ResNet-50 (teacher). Сравните accuracy student с/без дистилляции.

```
import torchvis
import models as models

# Teacher: большая предобученная модель
teacher = models.resnet50(weights=DEFAULT)
teacher.fc = nn.Linear(2048, 10) # 10 классов CIFAR-10
# (предварительно дообучить teacher на CIFAR-10)
teacher.eval() # замораживаем teacher

# Student: компактная модель
student = models.mobilenet_v2(weights=None) # без предобучения
student.classifier[1] = nn.Linear(1280, 10)

# Обучение student с дистилляцией
optimizer = torch.optim.Adam(student.parameters(), lr=1e-3)
for epoch in range(10):
    student.train()
    for images, labels in train_loader:
        with torch.no_grad():
            t_logits = teacher(images) # «мягкие» метки
            s_logits = student(images)
            loss = distillation_loss(
                s_logits, t_logits, labels, T=4.0, alpha=0.7)
        optimizer.zero_grad()
        loss.backward(); optimizer.step()
```

Тема 4. TPOT: генетическое программирование

TPOT (Tree-based Pipeline Optimization Tool) — библиотека AutoML, использующая генетическое программирование (GP) для автоматического конструирования ML-пайплайнов. Разработана в Университете Пенсильвании (Olson et al., 2016). TPOT представляет пайплайн как дерево, в котором узлы — операторы `scikit-learn`: предобработка, отбор признаков, классификатор или регрессор.

Алгоритм работает следующим образом. На первом шаге генерируется начальная популяция случайных пайплайнов (деревьев). Каждый пайплайн оценивается на кросс-валидации. Далее применяются генетические операторы: селекция (турнирная), скрещивание (обмен поддеревьями) и мутация (замена узла). Процесс повторяется заданное число поколений, при этом популяция эволюционирует в направлении более качественных пайплайнов.

Ключевое преимущество TPOT — возможность экспорта найденного пайплайна в воспроизводимый Python-скрипт. Это обеспечивает полную прозрачность: исследователь видит каждый шаг преобразования и может модифицировать его. Кроме того, TPOT поддерживает пользовательские конфигурации пространства поиска, что позволяет ограничить набор рассматриваемых алгоритмов.

Тема 5. Pruning в PyTorch

код

```
import torch.nn.utils.prune as prune
import torchvision.models as models

model = models.resnet18(weights=DEFAULT)

# 1. Unstructured L1 pruning: обнуление 50% мелких весов
for name, module in model.named_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.l1_unstructured(
            module, name='weight',
            amount=0.5) # доля удаляемых весов

# Подсчёт степени разрежённости (sparsity)
total, zeros = 0, 0
for p in model.parameters():
    total += p.numel()
    zeros += (p == 0).sum().item()
print(f'Sparsity: {zeros/total*100:.1f}%')

# 2. Structured pruning: удаление целых фильтров
prune.hook_structured(
    model.conv1, name='weight',
    amount=0.3, # удалить 30% фильтров
    n=1, # L1-норма для ранжирования
    dim=0) # по оси выходных каналов

# Сделать pruning перманентным (удалить маски)
for name, module in model.named_modules():
    if isinstance(module, torch.nn.Conv2d):
        prune.remove(module, 'weight')
```

Тема 5. Задача В: итеративный pruning ResNet

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Примените L1 unstructured pruning к ResNet-18 (CIFAR-10) с порогами 30%, 50%, 70%, 90%. Постройте график accuracy vs. sparsity.

```
import torch.nn as nn
import torch.nn.utils.prune as prune

results = [] # (sparsity, accuracy)

for amount in [0.3, 0.5, 0.7, 0.9]:
    # Копия базовой обученной модели
    model_pruned = copy.deepcopy(trained_model)

    # Применяем pruning к каждому Conv2d слою
    for name, module in model_pruned.named_modules():
        if isinstance(module, torch.nn.Conv2d):
            prune_l1_unstructured(
                module, weight, amount=amount)
            prune.remove(module, weight) # перманентный

    # Оценка accuracy после pruning (без дообучения)
    acc = evaluate(model_pruned, test_dl)
    results.append((amount*100, acc))
    print(f'Sparsity {amount*100:0f}% : acc={acc:4f}')

# Визуализация: accuracy vs sparsity
import matplotlib.pyplot as plt
sp, ac = zip(*results)
plt.plot(sp, ac, b-); plt.xlabel('Sparsity %')
plt.ylabel('Accuracy'); plt.title('Pruning trade-off')
plt.savefig('pruning_tradeoff.png', dpi=150)
```

Тема 6. Quantization: снижение разрядности

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Quantization — представление весов и активаций модели в типах данных с меньшей разрядностью: FP32 → INT8 (в 4 раза меньше памяти) или FP16 (в 2 раза). Это ускоряет inference и уменьшает размер модели без архитектурных изменений.

Типы квантизации: (1) Post-Training Quantization (PTQ) — применяется к уже обученной модели. Быстро, не требует переобучения. Калибровочный набор (100–1000 изображений) используется для оценки диапазонов активаций. (2) Quantization-Aware Training (QAT) — встраивание «фейковых» квантизаторов в граф вычислений во время обучения. Модель адаптируется к квантизационному шуму, что даёт лучшее качество (+0.5–1% accuracy vs. PTQ).

Форматы: INT8 (стандарт для CPU/NPU), FP16/BF16 (GPU-inference, автоматическое mixed-precision через torch.cuda.amp), INT4/NF4 (для LLM, QLoRA).

Инструменты: torch.quantization (PyTorch), TensorRT (NVIDIA), ONNX Runtime, OpenVINO (Intel), Core ML (Apple).

Тема 5. Auto-sklearn: пример использования

код

```
import autosklearn.classification
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, classification_report

# Загрузка данных (диагностика рака молочной железы)
X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Инициализация Auto-sklearn с временным бюджетом
automl = autosklearn.classification\
    .AutoSklearnClassifier(
        time_left_for_this_task=180, # общий бюджет: 3 минуты
        per_run_time_limit=30,      # лимит на одну модель
        ensemble_size=10,           # размер ансамбля
        metric=autosklearn.metrics.f1,
        seed=42)

automl.fit(X_tr, y_tr) # запуск AutoML

# Оценка результатов
y_pred = automl.predict(X_te)
print(f'AutoML F1: {f1_score(y_te, y_pred):.4f}')
print(classification_report(y_te, y_pred))

# Состав итогового ансамбля (какие модели и их веса)
print(automl.show_models())
```

Тема 5. Задача Б: Auto-sklearn на Digits

ПРАКТИЧЕСКАЯ ЗАДАЧА

📄 **Задание:** На Digits dataset обучите Auto-sklearn (бюджет 120с). Сравните F1-макро с базовым RandomForest. Выведите состав ансамбля.

```
import autosklearn.classification
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score
from sklearn.ensemble import RandomForestClassifier

# Digits: 64 признака (пиксели 8×8), 10 классов (цифры 0-9)
X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

# Базовый RF для сравнения
rf = RandomForestClassifier(random_state=42).fit(X_tr, y_tr)
print(f'RF F1-macro: {f1_score(y_te, rf.predict(X_te),
    average="macro"):.4f}')

# Auto-sklearn
aml = autosklearn.classification\
    .AutoSklearnClassifier(
    time_left_for_this_task=120, per_run_time_limit=20,
    seed=42)
aml.fit(X_tr, y_tr)
print(f'AutoML F1-macro: {f1_score(y_te, aml.predict(X_te),
    average="macro"):.4f}')
```

Тема 6. H2O AutoML: масштабируемая платформа

H2O AutoML — компонент распределённой ML-платформы H2O.ai, реализующий автоматический поиск лучшей модели с минимальной конфигурацией. В отличие от TPOT и Auto-sklearn, H2O использует фиксированный набор алгоритмов: GLM, Random Forest, GBM, XGBoost, Deep Learning (многослойный перцептрон), а также строит Stacked Ensemble из лучших обученных моделей.

Архитектура H2O основана на распределённом вычислительном фреймворке, хранящем данные в оперативной памяти в колоночном формате (H2OFrame). Это обеспечивает обработку наборов данных, превышающих объём памяти одной машины. AutoML-модуль последовательно обучает модели в рамках заданного временного бюджета и формирует leaderboard — ранжированный список по метрике.

Stacked Ensemble — ансамбль, в котором предсказания базовых моделей используются как признаки для мета-модели (по умолчанию — GLM с L1-регуляризацией). H2O строит два вида ансамблей: Best-of-Family (по одной лучшей модели каждого типа) и All Models (все обученные модели). Данный подход обеспечивает наивысшее качество за счёт комбинирования разнородных моделей.

Тема 6. H2O AutoML: пример использования

код

```
import h2o
from h2o.automl import H2OAutoML
from sklearn.datasets import fetch_california_housing

# Инициализация JVM-кластера H2O
h2o.init()

# Загрузка данных и конвертация в H2OFrame
data = fetch_california_housing(as_frame=True).frame
hf = h2o.H2OFrame(data)
train, test = hf.split_frame(ratios=[0.8], seed=42)

target = 'MedHouseVal' # целевая переменная
features = [c for c in hf.columns if c != target]

# Запуск AutoML с временным бюджетом
aml = H2OAutoML(
    max_runtime_secs=120, # бюджет 2 минуты
    seed=42,
    sort_metric='RMSE'    # сортировка leaderboard
)
aml.train(x=features, y=target, training_frame=train)

# Результаты: leaderboard и метрики лидера
print(aml.leaderboard.head(10)) # топ-10 моделей
perf = aml.leader.model_performance(test)
print(f'RMSE: {perf.rmse():.4f}, R2: {perf.r2():.4f}')
```

Тема 6. Задача В: регрессия California Housing

ПРАКТИЧЕСКАЯ ЗАДАЧА

📄 **Задание:** Запустите H2O AutoML на 180 секунд. Сравните RMSE лидера и Stacked Ensemble. Выведите важность признаков лидера.

```
import h2o
from h2o.automl import H2OAutoML
from sklearn.datasets import fetch_california_housing

h2o.init() # запуск кластера
data = fetch_california_housing(as_frame=True).frame
hf = h2o.H2OFrame(data)
train, test = hf.split_frame(ratios=[0.8], seed=42)

# Увеличенный бюджет для лучшего результата
aml = H2OAutoML(max_runtime_secs=180, seed=42,
                 sort_metric='RMSE')
aml.train(
    x=[c for c in hf.columns if c != 'MedHouseVal'],
    y='MedHouseVal', training_frame=train)

# Анализ leaderboard
print(aml.leaderboard.head(10))
perf = aml.leader.model_performance(test)
print(f'RMSE: {perf.rmse():.4f}, R2: {perf.r2():.4f}')

# Важность признаков лидера
if hasattr(aml.leader, 'varimp'):
    print(aml.leader.varimp(use_pandas=True).head(8))
```

Тема 7. AutoML и несбалансированные данные

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Несбалансированность классов — одна из наиболее распространённых проблем при решении задач классификации в реальных приложениях: обнаружение мошенничества, медицинская диагностика, предиктивное обслуживание. Соотношение классов может составлять 1:100 и более, что приводит к смещению модели.

AutoML-системы предоставляют ряд механизмов для работы с дисбалансом. Во-первых, выбор метрики: accuracy неинформативна при дисбалансе, вместо неё используются F1-score, ROC-AUC, Average Precision или log_loss. Во-вторых, взвешивание классов: параметр `class_weight='balanced'` автоматически пересчитывает веса пропорционально обратной частоте классов.

В-третьих, ресемплинг: оверсемплинг миноритарного класса (SMOTE, ADASYN) или андерсемплинг мажоритарного (RandomUnderSampler). Библиотека imbalanced-learn интегрируется с sklearn Pipeline. TPOT поддерживает кастомные операторы ресемплинга. H2O реализует встроенный параметр `balance_classes=True`, выполняющий оверсемплинг автоматически.

Тема 7. Стратегии ресемплинга: SMOTE

КОД

```
from imblearn.over_sampling import SMOTE
from imblearn.pipeline import Pipeline as ImbPipeline
from imblearn.combine import SMOTETomek
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import classification_report

# Пайплайн 1: SMOTE + RandomForest
# SMOTE генерирует синтетические объекты миноритарного класса
pipe_smote = ImbPipeline([
    ('smote', SMOTE(random_state=42)), # оверсемплинг
    ('clf', RandomForestClassifier(random_state=42))
])
pipe_smote.fit(X_tr, y_tr)
print('=== SMOTE + RF ===')
print(classification_report(y_te, pipe_smote.predict(X_te)))

# Пайплайн 2: SMOTETomek (оверсемплинг + очистка границ)
# Tomek Links удаляет пограничные объекты после SMOTE
pipe_combo = ImbPipeline([
    ('resample', SMOTETomek(random_state=42)),
    ('clf', RandomForestClassifier(
        class_weight='balanced', # доп. взвешивание
        random_state=42))
])
pipe_combo.fit(X_tr, y_tr)
print('=== SMOTETomek + Balanced RF ===')
print(classification_report(y_te, pipe_combo.predict(X_te)))
```

Тема 7. Задача Г: обнаружение мошенничества

📄 ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: На синтетическом наборе (5% fraud) обучите TPOT с метрикой `average_precision`. Сравните с моделью без учёта дисбаланса.

```
from sklearn.datasets import make_classification
from tpot import TPOTClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# Генерация несбалансированных данных: 95% норма / 5% fraud
X, y = make_classification(
    n_samples=2000, n_features=20,
    weights=[0.95, 0.05], # соотношение классов
    flip_y=0.01,          # шум в метках
    random_state=42)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=42)

# TPOT с метрикой для несбалансированных данных
tpot = TPOTClassifier(
    generations=5, population_size=20,
    scoring='average_precision', # Area Under PR Curve
    cv=5, random_state=42, verbosity=2)
tpot.fit(X_tr, y_tr)

# Отчёт: precision/recall по каждому классу
print(classification_report(y_te, tpot.predict(X_te)))
```

Тема 8. Интерпретируемость моделей AutoML

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Автоматизация построения моделей порождает проблему «чёрного ящика»: модель, выбранная AutoML-системой, может быть сложным ансамблем, внутренняя логика которого непрозрачна для исследователя.

Различают два уровня интерпретируемости. Глобальная интерпретация отвечает на вопрос: «Какие признаки в целом наиболее важны для модели?» Для этого используются Permutation Importance и SHAP Summary Plot. Локальная интерпретация объясняет конкретное предсказание: «Почему модель присвоила данному объекту класс 1?» — SHAP Waterfall, LIME.

SHAP (SHapley Additive exPlanations, Lundberg & Lee, 2017) основан на теории кооперативных игр Шепли. Каждому признаку присваивается вклад в отклонение предсказания от среднего. Свойства: аддитивность, локальная точность, согласованность. Для древесных моделей существует эффективный алгоритм TreeSHAP с полиномиальной сложностью.

Практически все AutoML-библиотеки совместимы с SHAP, так как возвращают обученный объект модели, доступный для анализа.

Тема 8. SHAP: глобальная и локальная интерпретация

КОД

```
import shap
import matplotlib.pyplot as plt
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split

# Подготовка данных и обучение модели
X, y = load_breast_cancer(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = GradientBoostingClassifier(
    random_state=42).fit(X_tr, y_tr)

# Создание SHAP-объяснителя (TreeSHAP для древесных моделей)
explainer = shap.Explainer(model, X_tr)
shap_values = explainer(X_te) # вычисление SHAP-значений

# Глобальная интерпретация: какие признаки важнее?
shap.summary_plot(shap_values, X_te, show=False)
plt.tight_layout()
plt.savefig('shap_global.png', dpi=150)

# Локальная интерпретация: почему модель так решила?
shap.plots.waterfall(shap_values[0], show=False)
plt.tight_layout()
plt.savefig('shap_local.png', dpi=150)
```

Тема 8. Задача Д: SHAP-анализ модели TPOT

ПРАКТИЧЕСКАЯ ЗАДАЧА

Задание: Обучите TPOT на Wine dataset. Извлеките финальную модель и постройте SHAP summary plot. Определите три наиболее важных признака.

```
import shap, matplotlib.pyplot as plt
from tpot import TPOTClassifier
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Обучение TPOT на Wine dataset
X, y = load_wine(return_X_y=True, as_frame=True)
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.2, random_state=42)

tpot = TPOTClassifier(
    generations=5, population_size=20,
    cv=5, random_state=42, verbosity=0)
tpot.fit(X_tr, y_tr)

# Извлечение финальной модели из пайплайна TPOT
final_model = tpot.fitted_pipeline_.steps[-1][1]

# SHAP-анализ извлечённой модели
explainer = shap.Explainer(final_model, X_tr)
shap_values = explainer(X_te)
shap.summary_plot(shap_values, X_te, show=False)
plt.tight_layout()
plt.savefig('shap_wine.png', dpi=150)
```

Тема 9. AutoML для задач регрессии

Пространство метрик

Задачи регрессии в контексте AutoML имеют ряд специфических особенностей. Пространство метрик: RMSE, MAE, R^2 , MAPE — каждая метрика предъявляет различные требования к модели. RMSE чувствительна к выбросам, MAE более робастна, R^2 нормализована относительно дисперсии, а MAPE удобна для интерпретации в процентах.

Валидация

Для временных рядов стандартная кросс-валидация неприменима, требуется TimeSeriesSplit или expanding window. Optuna позволяет реализовать произвольную схему валидации внутри целевой функции, что даёт максимальную гибкость.

Предобработка целевой переменной

Логарифмирование, Box-Cox или Yeo-Johnson трансформации для нормализации распределения. AutoML-системы (H2O, Auto-sklearn) могут включать такие трансформации в пространство поиска.

Практика

Комбинация Optuna + LightGBM/CatBoost является одним из наиболее эффективных подходов для табличных данных регрессии, сочетая скорость обучения с гибкостью настройки.

Тема 9. Optuna + LightGBM: регрессия

код

```
import optuna
import lightgbm as lgb
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import cross_val_score

# Загрузка данных о стоимости жилья в Калифорнии
X, y = fetch_california_housing(return_X_y=True)

def objective(trial):
    """Целевая функция: минимизация RMSE через CV."""
    params = {
        'n_estimators': trial.suggest_int('n', 100, 1000),
        'max_depth': trial.suggest_int('d', 3, 12),
        'learning_rate': trial.suggest_float('lr', .01, .3, log=True),
        'num_leaves': trial.suggest_int('lv', 20, 150),
        'reg_alpha': trial.suggest_float('a', 1e-3, 10, log=True),
        'reg_lambda': trial.suggest_float('l', 1e-3, 10, log=True),
    }
    model = lgb.LGBMRegressor(**params, random_state=42,
                              verbose=-1)
    # neg_RMSE -> инвертируем знак для минимизации
    return -cross_val_score(model, X, y, cv=5,
                            scoring='neg_root_mean_squared_error').mean()

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=80)
print(f'Best RMSE: {study.best_value:.4f}')
```

Тема 9. Задача Е: Optuna + CatBoost

ПРАКТИЧЕСКАЯ ЗАДАЧА

📌 **Задание:** Подберите гиперпараметры CatBoostRegressor для California Housing через Optuna. Сравните RMSE с LightGBM из предыдущего слайда.

```
import optuna
from catboost import CatBoostRegressor
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import cross_val_score

# Те же данные для корректного сравнения
X, y = fetch_california_housing(return_X_y=True)

def objective(trial):
    """Оптимизация CatBoost через Optuna."""
    params = {
        'iterations': trial.suggest_int('it', 100, 800),
        'depth': trial.suggest_int('d', 4, 10),
        'learning_rate': trial.suggest_float('lr', .01, .3, log=True),
        'l2_leaf_reg': trial.suggest_float('l2', 1e-2, 10, log=True),
    }
    model = CatBoostRegressor(**params, random_state=42,
                              verbose=0)
    return -cross_val_score(model, X, y, cv=5,
                            scoring='neg_root_mean_squared_error').mean()

study = optuna.create_study(direction='minimize')
study.optimize(objective, n_trials=60)
print(f'CatBoost best RMSE: {study.best_value:.4f}')
```

Тема 10. Ограничения и лучшие практики AutoML

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Несмотря на прогресс, AutoML имеет принципиальные ограничения.

Вычислительная стоимость

Полный перебор пространства пайплайнов требует обучения сотен и тысяч моделей. Для крупных наборов данных (>100 000 объектов) время работы может составлять часы. Стратегии смягчения: ограничение бюджета, ранняя остановка (pruning), подвыборки.

Область применимости

AutoML оптимизирован для табличных данных. Для компьютерного зрения, NLP и временных рядов эффективнее специализированные подходы (NAS, AutoGluon-TimeSeries).

Воспроизводимость

Стохастическая природа поиска означает, что повторные запуски могут давать различные пайплайны. Фиксация `random_state`, логирование в MLflow/W&B и валидация на held-out выборке обязательны.

Экспертное знание

Необходимость экспертного знания предметной области для корректной постановки задачи, выбора метрики и валидации результатов. AutoML автоматизирует технические этапы, но не заменяет аналитика.

Интерпретируемость

Сложные ансамбли труднее объяснять заинтересованным сторонам. SHAP и LIME частично решают эту проблему.